

Aperture — Technical Architecture (Part 3)

Version: v1.0

Document Type: Technical Architecture

21. Infrastructure Philosophy

Infrastructure should support the application without becoming the application.

One of the most common mistakes in portfolio projects is introducing production-scale infrastructure before there is a production-scale problem.

Aperture intentionally evolves through infrastructure stages.

Stage 1

Developer Machine

↓

Stage 2

Single Cloud Deployment

↓

Stage 3

Production Infrastructure

↓

Stage 4

Scalable Cloud Platform

Each stage builds upon the previous one.

22. Containerization

Decision

Docker

Why?

Requirements:

- identical development environments
- reproducible builds
- simple onboarding
- deployment consistency

Docker allows the application to behave the same on:

- macOS
- Windows
- Linux
- CI/CD
- Cloud

without requiring developers to manually configure every dependency.

Alternatives

Native installation

Virtual Machines

Podman

Decision

Docker offers the best balance of:

- adoption
 - documentation
 - tooling
 - ecosystem
-

Docker Compose

Docker Compose orchestrates:

- frontend
- backend
- PostgreSQL
- Redis
- supporting services

The goal is:

```
docker compose up
```

Everything works.

No additional setup.

23. CI/CD Strategy

Decision

GitHub Actions

Every pull request should automatically execute:

- formatting
- linting
- type checking
- unit tests
- integration tests
- frontend build
- backend build
- Docker build

Only passing code can be merged.

Deployment Pipeline

Developer

↓

Git Push

↓

GitHub

↓

GitHub Actions

↓

Quality Gates

↓

Deployment

↓

Production

Why GitHub Actions?

Advantages

- native GitHub integration
- excellent ecosystem
- easy maintenance
- interview familiarity

Alternatives

- Jenkins
- CircleCI
- GitLab CI

GitHub Actions provides the simplest solution for this project's scale.

24. Infrastructure as Code

Decision

Terraform

Infrastructure should never be configured manually.

Instead, infrastructure becomes version-controlled.

Examples

- databases
- networking
- storage
- compute
- monitoring

Benefits

- repeatability
 - version history
 - disaster recovery
 - reproducibility
-

25. Cloud Strategy

Cloud adoption follows incremental stages.

Phase 1

Vercel

Frontend hosting

Fast global CDN

Simple deployments

Backend

Container deployment

Managed PostgreSQL

Managed Redis

Phase 2

Cloudflare

Adds:

- caching
 - edge routing
 - DDoS protection
 - performance improvements
-

Phase 3

AWS

Introduce

- S3
- CloudFront
- RDS
- ECS
- ElastiCache

only when required.

Why Not AWS First?

AWS provides flexibility but introduces operational complexity.

For a portfolio project, managed platforms enable faster product development while preserving a migration path.

26. Storage Strategy

Different types of data belong in different storage systems.

Relational Data



PostgreSQL

Cached Data



Redis

Images



TMDb CDN



Later



Object Storage

Logs



Logging Platform

Embeddings



PostgreSQL

↓

Later

↓

Dedicated Vector Store

27. Security Architecture

Security is treated as a core engineering concern rather than an afterthought.

Authentication

JWT

OAuth

Refresh Tokens

Authorization

Role-based permissions

Ownership checks

Resource-level authorization

Validation

Every external request is validated.

No client input is trusted.

Secrets

Never committed to source control.

Managed through environment variables and cloud secret management.

Rate Limiting

Protect authentication and search endpoints.

Security Headers

HTTPS

CORS

Content Security Policy

Secure Cookies

HSTS

Future

Audit logging

Two-factor authentication

Device management

28. Observability

The application should explain what it is doing.

Three pillars:

Logs

Metrics

Tracing

Logging

Structured logging.

Every request includes:

- request ID
- user ID (when available)
- latency
- endpoint
- status

Logs should be machine-readable.

Metrics

Track

- API latency
 - cache hit rate
 - search latency
 - recommendation latency
 - database response time
 - background job duration
-

Tracing

Future

Distributed tracing becomes relevant only after multiple independently deployed services exist.

29. Monitoring

Application health should be observable.

Health Checks

Readiness

Liveness

Database connectivity

Redis connectivity

TMDb availability

Search availability

Dashboards

Infrastructure

Application

Database

Recommendation pipeline

Search

Alerts

Errors

Latency

Deployment failures

Background job failures

30. Performance Engineering

Performance is approached systematically.

Never optimize prematurely.

Instead:

Measure

↓

Identify



Optimize



Measure Again

Optimization Priorities

Database



Caching



Queries



Images



Frontend Rendering



Algorithms

Avoid optimizing code that is not a bottleneck.

31. Testing Strategy

Testing is treated as part of feature development.

Every feature should include appropriate tests.

Unit Tests

Business logic

Integration Tests

Database

API

External services

API Tests

Endpoint behavior

Validation

Authentication

End-to-End Tests

Critical user journeys.

Examples

- Login
 - Rating a movie
 - Writing a review
 - Creating a list
-

Load Tests

Introduced before production launch.

32. Documentation Strategy

Documentation evolves alongside the codebase.

Every major change should update:

PRD

Roadmap

Architecture

LLD

Database

Relevant ADRs

Documentation should never become an afterthought.

33. Deployment Strategy

Development

↓

Staging

↓

Production

Every deployment should be automated.

Manual deployments are avoided wherever possible.

Rollback Strategy

Every deployment should be reversible.

Previous container images remain available for rollback.

34. Cost Optimization

Operational simplicity often reduces cost more effectively than aggressive infrastructure optimization.

Examples:

Use managed services until scale justifies self-hosting.

Avoid idle infrastructure.

Cache selectively.

Profile before upgrading hardware.

35. Technology Evolution Roadmap

Technology choices should evolve with the product.

Example

Search

PostgreSQL

↓

OpenSearch

↓

Semantic Search

Recommendations

Metadata

↓

Collaborative Filtering

↓

Embeddings



Hybrid Ranking

Infrastructure

Single Container



Managed Services



Cloud Platform



Selective Service Extraction

The architecture intentionally avoids "big bang" migrations.

Every transition should be incremental.

36. Engineering Principles

Every future technology proposal should answer the following questions:

1. What problem does this solve?
2. Why is the current solution insufficient?
3. Why is this technology preferable to alternatives?
4. What operational complexity does it introduce?
5. How will success be measured?

If these questions cannot be answered convincingly, the proposal should be reconsidered.

37. Technical Architecture Summary

Aperture's technical architecture emphasizes engineering discipline over technological novelty.

The platform adopts modern, widely used technologies that complement one another and are introduced progressively as the product evolves.

The resulting system is intentionally:

- maintainable
- scalable
- observable
- secure
- testable
- interview-ready

Rather than optimizing for hypothetical future scale, the architecture grows alongside real product requirements, preserving both developer productivity and architectural clarity throughout the project's lifecycle.